

Walchand College of Engineering, Sangli

Department of Computer Science and Engineering

Class: Final Year (Computer Science and Engineering)

Year: 2024-25

Semester: 1

Course: High Performance Computing Lab

Practical No. 5

Exam Seat No:

Title of practical: Implementation of OpenMP programs.

Implement following Programs using OpenMP with C:

1. Implementation of Matrix-Matrix Multiplication.
2. Implementation of Matrix-scalar Multiplication.
3. Implementation of Matrix-Vector Multiplication.
4. Implementation of Prefix sum.

Problem Statement 1: Implementation of Matrix-Matrix Multiplication.

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ ./ps1
Matrix-Matrix Multiplication completed in 0.046450 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$
```

Code :

```

int main() {
    static int A[N][N], B[N][N], C[N][N];
    int i, j, k;

    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            A[i][j] = rand() % 10;
            B[i][j] = rand() % 10;
            C[i][j] = 0;
        }
    }

    double st = omp_get_wtime();

    #pragma omp parallel for private(i, j, k) shared(A, B, C)
    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            for (k = 0; k < N; k++)
            {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    double end = omp_get_wtime();
    printf("Matrix-Matrix Multiplication completed in %f seconds\n", end - st);

    return 0;
}

```

Information:

The matrix multiplication requires $O(N^3)$ operations. For large N , it becomes very slow in sequential execution.

OpenMP allows us to parallelize loops so that multiple threads can work on different rows or elements of the result simultaneously.

Analysis:

- The parallelized version significantly reduces computation time compared to a sequential approach.
- As matrix size increases, the benefit of parallel execution grows because more computation is distributed among available CPU cores.
- For $N=500$, the computation took only ~ 0.046 seconds, demonstrating the efficiency of OpenMP parallelization.

Problem Statement 2:

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ g++ -fopenmp ps2.cpp -o ps2
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ ./ps2
Matrix-Scalar Multiplication completed in 0.004100 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$
```

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define R 500
#define C 500

int main() {
    static int A[R][C];
    int scalar = 5;

    int i, j;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            A[i][j] = rand() % 10;

    double st = omp_get_wtime();

    #pragma omp parallel for shared(A, scalar)
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            A[i][j] *= scalar;

    double end = omp_get_wtime();
    printf("Matrix-Scalar Multiplication completed in %f seconds\n", end - st);

    return 0;
}
```

Information:

The aim of this experiment is to implement Matrix-Scalar Multiplication using OpenMP and analyze its performance. The focus is on understanding how simple mathematical operations on matrices can be parallelized to achieve faster execution times.

Matrix-Scalar multiplication is a basic linear algebra operation.

Given a matrix A ($m \times n$) and a scalar value k, the multiplication is performed as:

$$C[i][j] = A[i][j] \times k$$

Here, every element of the matrix is multiplied independently by the scalar.

This operation is highly parallelizable, because each element of the matrix can be computed independently of others.

Analysis:

The computation involves only one multiplication per matrix element, making it less intensive compared to matrix–matrix multiplication.

Because each element is updated independently, the parallelization is straightforward and highly efficient.

Even for large matrices, the execution time is very low - **0.004100 seconds**

Problem Statement 3:

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ g++ -fopenmp ps3.cpp -o ps3
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ ./ps3
Matrix-Vector Multiplication completed in 0.00361806 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$
```

Code -

```
#define N 500

int main() {
    static int A[N][N], x[N], y[N];
    int i, j;

    for (i = 0; i < N; i++) {
        x[i] = rand() % 10;
        for (j = 0; j < N; j++)
            A[i][j] = rand() % 10;
    }

    double st = omp_get_wtime();
    // Matrix-Vector Multiplication
    #pragma omp parallel for private(j) shared(A, x, y)
    for (i = 0; i < N; i++) {
        y[i] = 0;
        for (j = 0; j < N; j++)
            y[i] += A[i][j] * x[j];
    }

    double end = omp_get_wtime();
    cout << "Matrix-Vector Multiplication completed in "
        << (end - st) << " seconds\n";

    return 0;
}
```

Information:

The aim of this experiment is to implement Matrix-Vector Multiplication using OpenMP and study its performance improvement compared to a sequential approach.

Matrix-Vector multiplication is a common operation in linear algebra, scientific computing, and machine learning.

This requires $O(N^2)$ operations. Each element of the result vector $y[i]$ depends only on row i of A and the vector x , which makes the operation naturally parallelizable.

Analysis:

The sequential version of matrix-vector multiplication would require processing each row one after another.

With OpenMP, the computation is divided among available CPU cores, so multiple rows are processed in parallel.

This significantly reduces execution time, especially for larger matrix sizes.

For $N = 500$, the multiplication was completed in 0.00361806 seconds, showing clear performance efficiency with OpenMP.

Problem Statement 4:

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ g++ -fopenmp ps4.cpp -o ps4
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment5$ ./ps4
Prefix sum completed in 0.012805 seconds
```

Code :

```
int main() {
    int N = 1000000;
    int arr[N], pre[N];
    for (int i = 0; i < N; i++)
        arr[i] = 1;

    double st = omp_get_wtime();

    pre[0] = arr[0];
    #pragma omp parallel for
    for (int i = 1; i < N; i++)
        pre[i] = pre[i - 1] + arr[i];

    double end = omp_get_wtime();
    printf("Prefix sum completed in %f seconds\n", end - st);

    return 0;
}
```

Information:

Each element of the prefix sum depends on the previous element's result.

This dependency makes parallelization tricky. In our implementation, we still used parallel for to demonstrate OpenMP, though in practice prefix sum is usually computed with special two-phase algorithms to avoid dependencies.

Output - 0.012805 seconds

Therefore.. even for a large array size of one million elements, the computation is completed efficiently.

Github Link: [🌐 HPCL/Assignment5 at main · Somzee5/HPCL](#)